Mohammad Fauzan P.D.

26417020159

TI – 1C / 19

Percobaan 1 :

1. Sebutkan dan tunjukkan masing-masing komponen perulangan FOR pada kode program Percobaan 1!
2. Mengapa variabel **tertinggi** diinisialisasi 0 dan **terendah** diinisialisasi 100? Apa yang terjadi jika variabel tertinggi diinisialisasi 100 dan terendah diinisialisasi 0?
3. Jelaskan fungsi dan alur kerja dari potongan kode berikut!

```
if (nilai > tertinggi) {
    tertinggi = nilai;
}
if (nilai < terendah) {
    terendah = nilai;
}
```

4. Modifikasi kode program sehingga terdapat perhitungan untuk menentukan berapa mahasiswa yang lulus dan yang tidak lulus berdasarkan batas kelulusan (nilai minimal 60). Tampilkan jumlah mahasiswa lulus dan tidak lulus setelah menampilkan nilai tertinggi dan terendah!

Jawaban :

1. **Inisialisasi (int i = 1):** Membuat dan memberi nilai awal variabel pengontrol perulangan (i) dengan angka 1. Variabel ini menandakan mahasiswa ke-1.
   Kondisi / Syarat Perulangan (i <= 10): Batasan atau syarat agar perulangan terus berjalan. Selama nilai i kurang dari atau sama dengan 10, blok kode di dalam perulangan akan terus dieksekusi.
   Update / Penambahan (i++): Perintah untuk menaikkan nilai i sebesar 1 setiap kali satu putaran perulangan selesai (increment).
2. **Mengapa diinisialisasi tertinggi = 0 dan terendah = 100?**
   tertinggi = 0: Karena nilai mahasiswa paling kecil yang mungkin dimasukkan adalah 0, maka nilai awal diset 0 agar setiap nilai baru yang dimasukkan (yang biasanya ≥ 0) bisa langsung menggantikan posisi awal ini jika lebih besar.
   terendah = 100: Karena nilai mahasiswa paling tinggi umumnya adalah 100, maka nilai awal diset 100 agar setiap nilai baru yang dimasukkan (yang biasanya ≤ 100) bisa langsung lebih kecil dan menggantikan nilai awal ini.
   Apa yang terjadi jika dibalik (tertinggi = 100 dan terendah = 0)?
   Jika tertinggi = 100, maka jika semua nilai mahasiswa yang dimasukkan di bawah 100 (misalnya maksimal hanya 95), variabel tertinggi akan tetap bernilai 100 (salah/tidak akurat).
   Jika terendah = 0, maka jika semua nilai mahasiswa di atas 0 (misalnya paling kecil adalah 50), variabel terendah akan tetap bernilai 0 (salah/tidak akurat).

3. **Fungsi:** Potongan kode ini berfungsi untuk melakukan *screening* atau pengecekan secara dinamis guna mencari nilai maksimum (tertinggi) dan nilai minimum (terendah) dari sekumpulan nilai yang diinputkan pengguna.

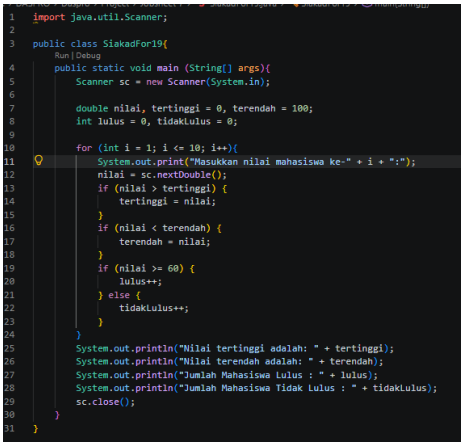
   Alur Kerja:

   Setiap kali pengguna memasukkan sebuah `nilai`, program akan mengecek kondisi pertama: `Apakah nilai yang baru dimasukkan lebih besar dari variabel tertinggi saat ini?`

   Jika **ya**, maka nilai variabel `tertinggi` akan diperbarui dengan `nilai` yang baru saja dimasukkan. Jika **tidak**, nilai `tertinggi` tetap tidak berubah.

   Selanjutnya, program mengecek kondisi kedua: `Apakah nilai yang baru dimasukkan lebih kecil dari variabel terendah saat ini?`

   Jika **ya**, maka nilai variabel `terendah` akan diperbarui dengan `nilai` tersebut. Jika **tidak**, nilai `terendah` tetap tidak berubah.

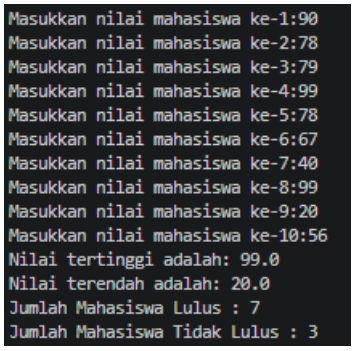
4. 

```
import java.util.Scanner;

public class SiakadFor19{
    Run | Debug
    public static void main (String[] args){
        Scanner sc = new Scanner(System.in);

        double nilai, tertinggi = 0, terendah = 100;
        int lulus = 0, tidakLulus = 0;

        for (int i = 1; i <= 10; i++){
            System.out.print("Masukkan nilai mahasiswa ke-" + i + ":");
            nilai = sc.nextDouble();
            if (nilai > tertinggi) {
                tertinggi = nilai;
            }
            if (nilai < terendah) {
                terendah = nilai;
            }
            if (nilai >= 60) {
                lulus++;
            } else {
                tidakLulus++;
            }
        }
        System.out.println("Nilai tertinggi adalah: " + tertinggi);
        System.out.println("Nilai terendah adalah: " + terendah);
        System.out.println("Jumlah Mahasiswa Lulus : " + lulus);
        System.out.println("Jumlah Mahasiswa Tidak Lulus : " + tidakLulus);
        sc.close();
    }
}
```



```
Masukkan nilai mahasiswa ke-1:90
Masukkan nilai mahasiswa ke-2:78
Masukkan nilai mahasiswa ke-3:79
Masukkan nilai mahasiswa ke-4:99
Masukkan nilai mahasiswa ke-5:78
Masukkan nilai mahasiswa ke-6:67
Masukkan nilai mahasiswa ke-7:40
Masukkan nilai mahasiswa ke-8:99
Masukkan nilai mahasiswa ke-9:20
Masukkan nilai mahasiswa ke-10:56
Nilai tertinggi adalah: 99.0
Nilai terendah adalah: 20.0
Jumlah Mahasiswa Lulus : 7
Jumlah Mahasiswa Tidak Lulus : 3
```

Percobaan 2 :

1. Pada potongan kode berikut, tentukan maksud dan kegunaan dari sintaks berikut:

```
if (nilai < 0 || nilai > 100) {
    System.out.println(x:"Nilai tidak valid. Masukkan lagi nilai yang valid!");
    continue;
}
```

   a. **nilai < 0 || nilai > 100**
   b. **continue**
2. Mengapa sintaks **i++** dituliskan di akhir perulangan WHILE? Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?
3. Apabila jumlah mahasiswa yang dimasukkan adalah 19, berapa kali perulangan WHILE akan berjalan?
4. Modifikasi kode program sehingga apabila terdapat mahasiswa yang mendapat nilai A, program menampilkan pesan tambahan "Bagus, pertahankan nilainya"!

Jawaban :

1. **`nilai < 0 || nilai > 100`**
   Maksud & Kegunaan: Ini adalah kondisi pengecekan untuk memvalidasi input nilai. Operator `||` (OR) berarti kondisi bernilai benar jika **salah satu** syarat terpenuhi—yaitu jika nilai yang dimasukkan kurang dari `0` **atau** lebih besar dari `100`. Jika pengguna memasukkan nilai di luar rentang yang valid ini, maka blok kode di dalam `if` akan dijalankan.

   `Continue`
   Maksud & Kegunaan: Sintaks ini berfungsi untuk **menghentikan iterasi (putaran) saat ini secara langsung** dan melewatkan sisa baris kode di bawahnya pada putaran tersebut. Program akan langsung melompat kembali ke awal perulangan `while` untuk meminta input ulang tanpa menambah nilai `i` (sehingga mahasiswa ke- tersebut tetap diulang pemasukannya).

2. **Mengapa `i++` dituliskan di akhir perulangan WHILE?**
   Karena `i` digunakan sebagai penghitung (counter) jumlah mahasiswa yang sudah berhasil diproses. Dengan meletakkannya di akhir, `i++` hanya akan bertambah jika input nilai dari pengguna **valid** (tidak memicu `continue`). Jika ditaruh di awal atau sebelum validasi, ketika nilai tidak valid dan `continue` terpanggil, posisi `i` bisa melompati tahapan sehingga perhitungan jumlah mahasiswa menjadi kacau.

   Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?
   Jika pengguna memasukkan nilai yang tidak valid (misalnya `-5`), perintah `continue` akan dieksekusi sebelum `i++` tercapai? *Tergantung penempatannya*, jika ditaruh tepat di baris pertama sebelum validasi, nilai `i` akan bertambah terlebih dahulu. Namun, jika terjadi kesalahan input dan `continue` melompat ke atas, alur penambahan bisa tidak sinkron, atau jika ditaruh tepat sebelum `continue`, perulangan bisa mengalami *infinite loop* (perulangan tak berhingga) atau data indeks mahasiswa bergeser karena `i` terus bertambah meskipun inputnya ditolak.

3. Perulangan akan berjalan sebanyak **19 kali** (selama data yang dimasukkan valid). Kondisi perulangan adalah while (i < jml), di mana jml bernilai 19 dan i dimulai dari 0. Perulangan akan terus berputar dari i = 0 hingga i = 18 (total tepat 19 kali putaran).

4.



```
import java.util.Scanner;

public class SiakadWhile19 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int i = 0;

        System.out.print("Masukkan jumlah mahasiswa: ");
        int jml = sc.nextInt();

        while (i < jml) {
            System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ": ");
            int nilai = sc.nextInt();

            if (nilai < 0 || nilai > 100) {
                System.out.println("Nilai tidak valid, masukkan nilai antara 0-100");
                continue;
            }
            if (nilai > 80) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah A");
                System.out.println("Bagus Pertahankan Nilainya");
            } else if (nilai > 73) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah B+");
            } else if (nilai > 65) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah B");
            } else if (nilai > 60) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah C+");
            } else if (nilai > 50) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah C");
            } else if (nilai > 39) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah D");
            } else {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah E");
            }
            i++;
        }

        sc.close();
    }
}
```

```
Masukkan jumlah mahasiswa: 3
Masukkan nilai mahasiswa ke-1: 89
Nilai mahasiswa ke-1 adalah A
Bagus Pertahankan Nilainya
Masukkan nilai mahasiswa ke-2: 67
Nilai mahasiswa ke-2 adalah B
Masukkan nilai mahasiswa ke-3: 45
Nilai mahasiswa ke-3 adalah D
```

Percobaan 3 :

1. Pada penggunaan DO-WHILE ini, apabila nama pelanggan yang dimasukkan pertama kali adalah “batal”, maka berapa kali perulangan dilakukan?
2. Sebutkan kondisi berhenti yang digunakan pada perulangan DO-WHILE tersebut!
3. Apa fungsi dari penggunaan nilai **true** pada kondisi DO-WHILE?
4. Mengapa perulangan DO-WHILE tersebut tetap berjalan meskipun tidak ada komponen inisialisasi dan update?

Jawaban :

1. Perulangan akan dilakukan sebanyak **1 kali**.
   Penjelasan: Karena struktur do-while menjalankan blok kode di dalam bagian do terlebih dahulu minimal satu kali sebelum memeriksa kondisi di bagian bawah (while), maka meskipun input pertama langsung "batal", program akan tetap mengeksekusi baris tersebut, mendeteksi perintah break, dan langsung keluar dari perulangan.
2. Kondisi berhenti pada program ini tidak diletakkan langsung di dalam kurung while(...), melainkan dikendalikan menggunakan pernyataan kondisi di dalam blok if dengan perintah **break**:
   Java
   ```
   if (namaPelanggan.equalsIgnoreCase("batal")) {
     break;
   }
   ```
   Perulangan akan berhenti ketika pengguna memasukkan teks "batal" (tidak peduli huruf besar atau kecil karena menggunakan equalsIgnoreCase), yang kemudian memicu perintah break untuk keluar secara paksa dari perulangan do-while.
3. Nilai true pada while(true); berfungsi untuk membuat **perulangan tak berhingga (infinite loop)** secara sengaja.
   Hal ini dilakukan karena jumlah transaksi atau pelanggan yang akan dilayani sifatnya fleksibel/tidak ditentukan di awal, sehingga perulangan dibiarkan terus menerus berjalan sampai pengguna sendiri yang memutuskan untuk menghentikannya dengan mengetik "batal" (yang memicu perintah break).
4. Tidak adanya komponen **inisialisasi** (seperti int i = 0) dan **update** (seperti i++) karena perulangan ini tidak dikendalikan oleh penghitung angka (*counter* numerik), melainkan dikendalikan oleh **kejadian/input berbasis teks** (yaitu nama pelanggan).

Selama pengguna tidak mengetikkan kata "batal", program akan terus meminta input data baru secara dinamis di dalam blok do. Oleh karena itu, penambahan angka seperti i++ tidak diperlukan dalam kasus ini.

Tugas 1 :

```
Masukkan Jumlah Tiket Yang Dibeli
15
Total Diskon : 15.0%
Total Bayar : Rp637500.0

Masukkan Jumlah Tiket Yang Dibeli
20
Total Diskon : 15.0%
Total Bayar : Rp850000.0

Masukkan Jumlah Tiket Yang Dibeli
0
Transaksi Selesai
```

```java
import java.util.Scanner;

public class Tugas1_19{
    Run | Debug
    public static void main (String[] args){
        Scanner sc = new Scanner(System.in);

        int hargaTiket = 50000, totalTiket = 0, jumlahTiket, totalPenjualan = 0;
        double diskon, totalBayar;

        while(true) {
            System.out.println("Masukkan Jumlah Tiket Yang Dibeli");
            jumlahTiket = sc.nextInt();

            if (jumlahTiket == 0) {
                System.out.println("Transaksi Selesai");
                break;
            }

            if (jumlahTiket < 0) {
                System.out.println("Jumlah tiket tidak valid, silahkan input ulang");
                continue;
            }

            if (jumlahTiket > 10) {
                diskon = 0.15;
            } else if (jumlahTiket > 4) {
                diskon = 0.1;
            } else {
                diskon = 0;
            }

            totalBayar = jumlahTiket * hargaTiket;
            totalBayar = totalBayar - (totalBayar * diskon);

            totalPenjualan += totalBayar;
            totalTiket += jumlahTiket;

            System.out.println("Total Diskon : " + (diskon * 100) + "%");
            System.out.println("Total Bayar : Rp" + totalBayar);
            System.out.println("");

        }

        System.out.println("===== RESI TIKET =====");
        System.out.println("Total Tiket : " + totalTiket);
        System.out.println("Total Terjual : " + totalPenjualan);

        sc.close();
    }
}
```

Tugas 2 :

```java
import java.util.Scanner;

public class Tugas2_19{
    Run | Debug
    public static void main (String[] args){
        Scanner sc = new Scanner(System.in);

        int jenis, durasi, total = 0;

        do {
         System.out.println("Masukkan Jenis Mobil = 1, Motor = 2, Keluar = 0");
         jenis = sc.nextInt();

         if (jenis == 1 || jenis == 2) {
             System.out.println("Masukkan Lama Durasi Parkir (jam) : ");
             durasi = sc.nextInt();

             if (jenis == 1) {
                 total += durasi * 3000;
             } else {
                 total += durasi * 2000;
             }
         }
        } while (jenis != 0);

        System.out.println("Total Biaya Parkir: Rp" + total);
        sc.close();
    }
}
```

```
Masukkan Jenis Mobil = 1, Motor = 2, Keluar = 0
1
Masukkan Lama Durasi Parkir (jam) :
4
Masukkan Jenis Mobil = 1, Motor = 2, Keluar = 0
2
Masukkan Lama Durasi Parkir (jam) :
5
Masukkan Jenis Mobil = 1, Motor = 2, Keluar = 0
0
Total Biaya Parkir: Rp22000
```